

SQL Server Master Data Services (MDS) with practical examples so you can understand *what it is, why it is used, and how it is applied in real business scenarios.*

✅ What Is Master Data Services (MDS)?

Master Data Services (MDS) is a component of Microsoft SQL Server used for **Master Data Management (MDM)**. It helps organizations maintain a **single, clean, consistent, and authoritative version of key business data** such as:

- Customer data
- Product catalog
- Employees
- Vendors
- Chart of accounts
- Locations
- Assets
- Pricing master

MDS ensures that this *master data* is:

- ✓ Consistent
- ✓ Validated
- ✓ Versioned
- ✓ Approved
- ✓ Shared across systems (ERP, CRM, BI, etc.)

It includes:

- **MDS Database** (storage)
- **MDS Web Portal** (management UI)
- **Excel Add-in for MDS** (for business users)
- **Business Rules engine** (data quality rules)
- **Workflow approvals** (via SQL & SharePoint or Power Automate)
- **Versioning + Hierarchies**

🔗 Why Companies Use MDS

Organizations usually have the same data stored in many systems. For example:

Department	System	Stores "Product"?
Sales	CRM	Yes
Finance	ERP	Yes
Marketing	PIM	Yes
Inventory	WMS	Yes

Each system may store "Product" differently → causing conflicts.

- 👉 MDS provides **one centralized master definition** of "Product"
- 👉 All systems read the same master data
- 👉 No duplicates, no inconsistency, no confusion

Key Components of MDS

Component	Description
Models	Top-level container (e.g., Product Model)
Entities	Tables inside model (Products, Customers)
Attributes	Columns (Name, SKU, Category)
Members	Data records
Hierarchies	For grouping or taxonomy
Business Rules	Validation rules
Versions	Snapshots of data over time
Staging Tables	For automated data loading (ETL)

Real-Time Scenarios

Below are **industry-ready examples** that match real IT project usage.

Scenario 1: Product Data Centralization (Retail / E-Commerce)

Problem Before MDS

A retail company uses:

- SAP for ERP
- Salesforce for Sales
- Magento for E-commerce

Each system stores products differently:

System	Product Name	SKU	Category
SAP	LG TV 55"	LG55TV	Electronics
Salesforce	55-Inch LG Television	LG55-TV	TV
Magento	LG 55inch LED	LGTV55	LED

This causes:

- Wrong product data in reports
- Wrong SKUs shipped
- Duplicate products
- Incorrect taxation

Solution Using MDS

The company creates an **MDS Model: ProductMaster**

Entities:

- Product
- Category
- Brand

Business rules:

- SKU must be unique
- Product must have a valid category
- ProductName must start with BrandName
- Dimensions cannot be empty

Approval workflow:

- Marketing creates product →
- Supply Chain verifies →

- Finance approves pricing →
- Final version is published

Integration:

- Azure Data Factory loads data into MDS staging tables
- ERP/CRM/E-commerce consume final approved version

★ Outcome

- 100% consistent product catalog
- Reduction in returns due to correct SKU mapping
- Consistent website & POS data
- Faster onboarding of new products

Scenario 2: Customer Master Management (Banking / Insurance)

✗ Problem Before MDS

A bank has customer data from:

- Loan application system
- Mobile banking app
- Core banking system
- Credit card system

Duplicate customers exist:

- "John D"
- "John Doe"
- "J. Doe"
- "John Doe (VIP)"

✓ Solution Using MDS

Create **CustomerMaster** model with entities:

- Customer
- Address
- Customer Type

Rules:

- Email must be unique
- Phone number must be valid format
- Duplicate detection using fuzzy match
- VIP status must be approved manually

Workflows:

- Data Stewards review duplicates
- Compliance team approves status changes

Third-party systems:

- Call center system pulls latest 360-degree customer record
- Analytics platform uses same consistent customer dimension

★ Outcome

- Eliminated duplicate customers
- Accurate KYC data
- Improved fraud checks
- Clean customer data for analytics

Scenario 3: Reference Data Management (Finance / Manufacturing)

Reference Data = Common dropdown values used across systems.

Examples:

- Country list
- Currency list
- Departments
- Cost centers
- Product categories

Often, these become inconsistent across systems.

✓ Using MDS

Maintain a single master list of:

- Currency Codes (ISO standard)
- Country Codes (Alpha-3 / Alpha-2)
- Department hierarchy
- Chart of Accounts

Rules:

- CountryCode must be 2 characters
- IsActive cannot be empty
- Currency conversion factor must be valid

★ Outcome

- All systems use the same codes
- Fewer integration failures
- Standardized reporting

Scenario 4: Data Governance & Auditing (Healthcare)

A healthcare provider must track changes to critical data like:

- Doctors
- Departments
- Medical procedure codes
- Insurance plans

MDS provides:

- Versioning
- Audit history
- Role-based access
- Approval workflows

Regulatory compliance (HIPAA, ISO) becomes easier.

Scenario 5: Warehouse Dimensional Data (BI/Analytics)

BI systems like Power BI require:

- Clean dimensions (Product, Customer, Vendor, Region)

Most warehouses suffer from **slow-changing dimensions**.

MDS solves this by:

- Publishing versioned "Golden Dimension"
- Storing historical versions
- Allowing business users to modify hierarchies (e.g., moving a product to a different category)

Benefits of Using MDS

Technical Benefits

- Centralized master data
- API/Service integration
- ETL integration
- Root cause analysis
- Version control

Business Benefits

- Better reporting accuracy
- Improved customer experience
- Regulatory compliance
- Single version of truth across departments
- Reduced operational errors

Summary in One Line

Master Data Services ensures that your organization uses clean, correct, consistent master data across all applications, preventing confusion, errors, and duplicate data.

<https://www.sqldbachamps.com/>

✓ 1. MDS Project Workflow (End-to-End Real-Time)

Below is the **standard Master Data Services project workflow** followed in real IT implementations:

◆ Step 1: Identify Master Data Domains

Work with business + IT to determine which areas need MDM:

Examples:

- Product Master
- Customer Master
- Vendor Master
- Chart of Accounts
- Cost Centers
- Locations
- Pricing Master

Deliverables:

- Domain list
- Prioritization
- Problem summary

◆ Step 2: Data Profiling & Source Analysis

Use tools like SQL, Excel, Power BI, or Data Quality Services (DQS):

You check:

- Duplicate records
- Incomplete data
- Conflicting values
- Invalid values
- Missing relationship mappings

Deliverables:

- Profiling report
- Data issues list

◆ Step 3: MDM Requirements Gathering

Work with departments:

- Who creates data?
- Who approves?
- What validation rules apply?
- How systems integrate?
- What hierarchy is required?

Deliverables:

- Business rules
- Hierarchy requirements
- Approval workflows

◆ Step 4: Design MDS Model

You build MDS structure:

- **Model:** ProductMaster
- **Entities:** Product, Brand, Category
- **Attributes:** Name, SKU, Price, etc.
- **Domain-based attributes:** Category → Product
- **Hierarchies:** Category > Product

Deliverables:

- MDS data model
- Attribute definitions

◆ Step 5: Implement Business Rules

Examples:

- SKU must be unique
- Email must be valid
- Price cannot be negative
- Mandatory attributes must be filled

Types:

- Validation rules
- Default values
- Conditional rules
- Workflow triggering rules

◆ Step 6: Build Staging & Integration Layer

ETL loads data to MDS staging tables:

Tools:

- SSIS
- ADF (if cloud)
- Custom SQL jobs

Deliverables:

- Staging packages
- Sync processes

◆ Step 7: Create Workflows & Security

Role-based security:

- Data Stewards
- Approvers
- Contributors
- Auditors

Workflow approvals:

- Marketing → SCM → Finance → Publish

Deliverables:

- Security matrix
- Workflow documentation

◆ Step 8: Versioning & Publishing

Versions allow:

- Freeze & approve changes
- Rollback if needed
- Send golden master to downstream systems

◆ Step 9: Integration with Downstream Systems

MDS can publish data to:

- ERP (SAP/D365)
- CRM (Salesforce)
- E-commerce platforms
- Data warehouses
- Power BI

◆ Step 10: Support & Maintenance

- Monitor data quality
- Add new rules
- Extend entities
- Manage approvals

🔗 Final Workflow Summary Diagram (Conceptual)

Identify → Profile → Design → Build → Validate → Approve → Publish → Integrate → Maintain

✅ 2. Comparison: MDS vs Azure Purview vs Profisee

Here is a clear enterprise comparison:

◆ Overview Comparison

Feature	MDS	Azure Purview	Profisee
Type	Master Data Management	Data Governance / Catalog	Enterprise MDM
Deployment	On-prem	Cloud	Cloud + On-prem
Use Cases	Product/Customer Master	Data discovery + lineage	Complete enterprise MDM
Integrations	SQL, SSIS, Excel	Azure ecosystem	ERP, CRM, MDM, Cloud

◆ Functional Comparison

Capability	MDS	Purview	Profisee
MDM (Master Data Management)	✓	✗	✓✓✓
Data Cataloging	✗	✓✓✓	✓
Data Lineage	✗	✓✓✓	✓
Business Glossary	✗	✓✓✓	✓✓
Workflow Approvals	✓	✗	✓✓
Hierarchy Management	✓	✗	✓✓✓
Data Quality	Limited	✗	Strong
Cloud Native	✗	✓✓	Partial

◆ When to Use Which?

★ Use MDS If:

- You are on SQL Server / on-prem stack
- Need light-to-medium MDM
- Excel-based editing is preferred

★ Use Azure Purview If:

- You need enterprise data catalog
- Data lineage is crucial
- You want cloud-native metadata management

★ Use Profisee If:

- You need enterprise-grade MDM
- Multi-domain MDM required
- Complex workflows & data quality
- Cloud-first integration needed

✓ 3. Step-by-Step Guide to Set Up SQL Server MDS

Here is a clean, full setup guide:

✂ Step-by-Step Installation & Configuration

◆ Step 1: Install SQL Server

Ensure these features are included:

- Database Engine
- MDS Feature
- Integration Services (optional)

◆ Step 2: Run Master Data Services Configuration Manager

Launch:

- **Master Data Services Configuration**

Two steps here:

1. Database Configuration

- Create new MDS database
- Set MDS user credentials
- Configure collation

2. Web Application Configuration

- Create MDS website
- Bind to IIS
- Configure app pool identity

◆ Step 3: Set Up MDS User Permissions

Add user to:

- mds_model_reader
- mds_model_deployer
- mds_model_editor

◆ Step 4: Install MDS Excel Add-in

This allows business users to update master data directly.

◆ Step 5: Create Model

In MDS Web Portal:

- Create a new **Model**
- Define **Entities, Attributes, Hierarchies**

◆ Step 6: Create Business Rules

Examples:

- Email format validation
- Required attributes
- Relationships between entities

◆ Step 7: Load Data into Staging Tables

Tables:

- stg.<entity>_leaf
- stg.<entity>_member

Run stored procedure:

```
exec stg.udp_<entity>_leaf
```

◆ Step 8: Publish Version

After approval:

- Freeze version
- Validate
- Finalize
- Publish to downstream systems

📁 4. Real-Time MDS Interview Questions (With Answers)

Here are high-quality interview questions useful for SQL + MDM roles.

◆ Basic Questions

1. What is Master Data Services (MDS)?

A SQL Server component for Master Data Management (MDM) used to create and maintain a single version of truth for key business entities.

2. What are Entities in MDS?

They are tables representing master data, like Product, Customer, Vendor.

3. What is the purpose of Hierarchies?

To organize master data into logical parent-child structures.

4. What is a Model in MDS?

A container holding entities, attributes, rules, and hierarchies.

◆ Intermediate Questions

5. How does MDS enforce data quality?

Through:

- Business rules
- Domain attributes
- Validation
- Workflow approvals

6. What are staging tables used for?

For bulk data import using ETL (SSIS/ADF).

7. How does versioning work?

Each model can have multiple versions:

- Locked
- Open
- Validated
- Committed

Allows rollback & auditing.

8. What is a domain-based attribute?

A relationship attribute linked to another entity.

Example:

- Product.Category → Category entity

◆ Advanced / Real-Time Questions

9. How do you integrate MDS with ERP or CRM?

- Staging tables
- Web API / Service
- Export view
- SSIS packages
- Custom ETL jobs

10. Describe a real scenario where you used MDS.

Example answer:

Used MDS to centralize product master data across SAP, Salesforce, and e-commerce. Built business rules, workflows, and hierarchies, ensuring consistent SKU mapping. Published golden master to downstream systems via SSIS.

11. Can MDS handle multi-domain MDM?

Yes, but limited. Multiple models can be created but enterprise-grade multi-domain MDM is better handled by tools like Profisee.

12. Limitations of MDS?

- Not cloud-native
- No machine learning DQ
- No data lineage
- Limited workflow capabilities

<https://www.sqldbachamps.com/>

MDS mini-project:**1 Architecture Diagrams (Text + Simple Visuals)****A. Basic On-Prem MDS Architecture****Components:**

- Source systems: ERP, CRM, Files, Legacy DBs
- ETL: SSIS
- MDS:
 - MDS DB
 - MDS Web Portal
 - MDS Staging DB / tables
- Consumers: DWH, Reporting, Downstream apps

Flow:

```

[Source Systems]
| (Extract via SSIS)
v
[Staging DB / Landing]
| (Clean / Standardize)
v
[MDS Staging Tables]
| (MDS Import Procs)
v
[MDS Model (Entities, Rules, Hierarchies)]
| (Publish / Version)
v
[Subscriber Views / Export Tables]
| \
v  v
[Data Warehouse]  [ERP/CRM/Apps]
  
```

B. Hybrid: MDS + Azure Analytics

```

On-Prem:
ERP/CRM/Files
|
v
SSIS
|
v
MDS
|
v
MDS Export Views
|
v
Self-hosted IR
|
v
Azure:
ADF ---> Synapse / SQL DB ---> Power BI
  
```

2 MDS Hands-On Practice Exercises

You can implement these on a local SQL Server + MDS setup.

Exercise 1: Build a Product Master Model

1. **Create Model:** ProductMaster
2. **Entities:**
 - Product
 - Code (Identifier)
 - Name (Text)
 - Brand (Domain-based → Brand entity)
 - Category (Domain-based → Category entity)
 - ListPrice (Number)
 - IsActive (Boolean)
 - Brand
 - Code, Name
 - Category
 - Code, Name, ParentCategory (for hierarchy)
3. **Tasks:**
 - Load initial data via Excel Add-in.
 - Build a derived hierarchy Category > Product.
 - Create a version and commit it.

Exercise 2: Create Business Rules

On Product entity, implement:

- Rule 1: Name is mandatory.
- Rule 2: ListPrice > 0.
- Rule 3: If IsActive = FALSE, then ListPrice must be 0.
- Rule 4: Brand cannot be "Unknown".

Run **Validate Version** and fix records flagged as invalid.

Exercise 3: Use Staging Tables + Import

1. Identify staging table for Product entity:
stg.Product_Leaf (or similar).
2. Insert few rows into staging from INSERT script.
3. Execute stg.udp_Product_Leaf to load into MDS.
4. Verify records in MDS Web UI and through Excel Add-in.

Exercise 4: Publish to a Data Warehouse Table

1. Create Subscription View on Product entity (e.g., sv_ProductMaster).
2. Use a simple SSIS package or SELECT INTO to copy from sv_ProductMaster to DWH dimension table DimProduct.
3. Check that only **validated, committed** data appears.

3 Real-Time Project Documents (BRD / HLD / LLD) – Templates + Snippets

A. BRD – Business Requirements Document (Sample Outline)

Project: Enterprise Product Master Data Management using MDS

1. Business Background

- Multiple systems (SAP, Salesforce, E-com) maintain product data independently.
- Issues: inconsistent SKUs, wrong pricing, duplicate products.

2. Goals & Objectives

- Create a **single Product Master**.
- Enforce governance, approvals, and data quality.
- Provide trusted product data for reporting and operations.

3. In Scope

- Product, Brand, Category master.
- Integration with SAP & e-commerce.
- Excel-based maintenance for business users.

4. Out of Scope

- Customer master.
- Vendor master.

5. Business Requirements (examples)

BR-01 – All active products must have a unique SKU across the enterprise.

BR-02 – Each product must belong to exactly one category.

BR-03 – Prices must be validated and approved by Finance before going live.

BR-04 – Product data changes must be auditable (who/when/what).

6. Business Rules (high level)

- No duplicate SKUs.
- End-dated products cannot have new orders created.

7. Reporting Requirements

- Product catalog export.
- Change history report.

B. HLD – High-Level Design (Sample Outline)

1. Architecture Overview

- Describe diagram similar to section 1.
- Technology stack: SQL Server, SSIS, MDS, Power BI.

2. Data Domains & Entities

- Product, Brand, Category.
- Relationships & hierarchies.

3. Integration Design

- Upstream: SSIS jobs pull from SAP / Salesforce.
- Downstream: Subscription views feed DWH & apps.

4. Security & Roles

- Data Steward, Approver, Viewer.
- Row/attribute level security.

5. Versioning Strategy

- Monthly versions of ProductMaster.
- Process for promoting a version to “Production”.

6. Non-Functional Requirements

- Performance, availability, backup, DR.

C. LLD – Low-Level Design (Sample Outline)

1. Detailed Entity Definitions

Example – Product:

Attribute	Type	Required	Domain-based	Notes
Code	Text(50)	Yes	No	Business key (SKU)
Name	Text(255)	Yes	No	Product name
Brand	EntityRef	Yes	Yes	Ref → Brand entity
Category	EntityRef	Yes	Yes	Ref → Category entity

Attribute	Type	Required	Domain-based	Notes
ListPrice	Decimal	Yes	No	> 0 if IsActive = TRUE
IsActive	Boolean	Yes	No	TRUE/FALSE

2. Staging & ETL Mapping

Source System	Source Table	Staging Table	Transformation Notes
SAP	ZPRODUCTS	stg.Product_Leaf	Map SAP_SKU → Code, convert currency
Salesforce	Product2	stg.Product_Leaf	Resolve duplicate SKUs using match rules
E-com	dbo.Products	stg.Product_Leaf	Clean nulls, default IsActive = TRUE

3. Business Rule Config Details

- Rule names, conditions, actions (see next section).

4. Job Schedules

- Nightly import from sources.
- Early morning validation & version commit.
- Post-commit export to DWH.

4 SSIS Packages for MDS Integration (Conceptual Design)

You typically build **3 types of packages**:

Package 1: Load_MDS_Staging_Product.dtsx

Purpose: Load raw product data from multiple sources into stg.Product_Leaf.

Key Tasks:

1. Data Flow from each source system:
 - OLE DB / OData / flat file source.
 - Derived Column transformations (trim, uppercase, default values).
 - Lookup to identify duplicates / existing products.
 - Conditional Split for valid/invalid rows (invalid → error table).
2. OLE DB Destination → insert into stg.Product_Leaf.

Package 2: Trigger_MDS_Import_Product.dtsx

Purpose: Run MDS import stored procedure.

Steps:

1. Execute SQL Task:
2. EXEC stg.udp_Product_Leaf
3. @VersionName = 'ProductMaster_2025_01',
4. @LogFlag = 1;
5. Optionally, poll the MDS log table to ensure import success.

Package 3: Export_MDS_To_DWH_Product.dtsx

Purpose: Move committed master data to DWH.

Steps:

1. Execute SQL Task – ensure version status = “Committed”.
2. Data Flow:
 - Source: mdm.sv_ProductMaster (subscription view).
 - Destination: DWH.DimProduct.
 - Option: Slowly Changing Dimension (SCD) transformation.

You can parametrize version name, environment, and connection strings for DEV/UAT/PROD.

5 MDS Business Rule Examples

Let's define rules in business language *and* equivalent logic.

Rule 1 – SKU Must Be Unique

Entity: Product

Condition: On member save

Logic:

- If another record has the same Code and IsActive = TRUE, raise error.

(In MDS UI: set attribute Code as a unique identifier; combine with rule for IsActive if needed.)

Rule 2 – Price Must Be Positive for Active Products

Entity: Product

- **IF** IsActive = TRUE
- **AND** ListPrice <= 0
- **THEN** Validation fails with message:
"Active product must have a positive ListPrice."

Equivalent pseudo-SQL:

IF IsActive = 1 AND ListPrice <= 0

THROW 'Active product must have positive price';

Rule 3 – Mandatory Category

- **IF** Category IS NULL
- **THEN** Set Category to default "UNASSIGNED" **AND** raise a warning.

This is a **default + warning** pattern.

Rule 4 – Auto-Set Inactive Date

- **IF** IsActive changes from TRUE to FALSE
- **THEN** set InactiveDate = Today().

In MDS, this is implemented as a "Change-Based Action" rule.

Rule 5 – Email Format Check (Customer Entity)

- **IF** Email NOT LIKE pattern *@*. *
- **THEN** validation fails with message:
"Invalid email format."

(MDS has basic pattern validation; advanced checks can be done via staging / ETL.)

Rule 6 – Parent Category Must Be Valid

On Category entity:

- ParentCategory cannot be itself.
- Hierarchy depth limit: e.g., max 5 levels
(enforced via custom logic in staging/ETL + rule to avoid circular references).

1. End-to-end sample project design
2. Concrete SQL scripts (tables, sample data, “SSIS-friendly” SQL)
3. MDS-related staging + DWH scripts
4. A ready-to-use interview project story

1 Full Sample Project – “Product Master Hub”

Business Scenario

Company “RetailX” sells electronics through:

- ERP: ERPDB
- E-commerce: ECOMDB
- CRM: CRMDB

Each system has its own product list with inconsistent SKUs and names.

Goal: build **Product Master** using **SQL Server MDS** and feed a **data warehouse**.

High-Level Flow

1. Extract from source systems → **Landing/Staging DB**
2. Transform & load into **MDS staging tables**
3. Run MDS import → data becomes part of **MDS Product model**
4. Data stewards clean/approve/version in MDS
5. Export from MDS **subscription view** → **DWH DimProduct**

We'll focus on all the SQL parts around that.

2 Source & Landing Tables (Sample Scripts)

2.1 Source Product Tables (simulate source systems)

-- ERP Source

```
CREATE DATABASE ERPDB;
```

```
GO
```

```
USE ERPDB;
```

```
GO
```

```
CREATE TABLE dbo.Products_ERP (
    ERPProductID INT IDENTITY(1,1) PRIMARY KEY,
    SKU          VARCHAR(50) NOT NULL,
    ProductName  VARCHAR(200) NOT NULL,
    BrandName    VARCHAR(100) NULL,
    CategoryCode VARCHAR(50) NULL,
    ListPrice    DECIMAL(18,2) NULL,
    IsActive     BIT NOT NULL DEFAULT 1
);
```

```
INSERT INTO dbo.Products_ERP (SKU, ProductName, BrandName, CategoryCode, ListPrice, IsActive)
VALUES
```

```
('LG55TV', 'LG 55-Inch LED TV', 'LG', 'TV', 699.99, 1),
('MSGALAXY', 'Samsung Galaxy Phone', 'Samsung', 'MOB', 499.99, 1),
('OLDTV', 'Old CRT TV 21"', 'Generic', 'TV', 99.99, 0);
```

```
GO
```

-- E-commerce Source

```
CREATE DATABASE ECOMDB;
```

```
GO
```

```
USE ECOMDB;
```

```
GO
```

```
CREATE TABLE dbo.Products_ECOM (
    ECOMProductID INT IDENTITY(1,1) PRIMARY KEY,
    SKU          VARCHAR(50) NOT NULL,
```

```
Title      VARCHAR(200) NOT NULL,
Brand      VARCHAR(100) NULL,
Category   VARCHAR(50) NULL,
Price      DECIMAL(18,2) NULL,
ActiveFlag CHAR(1) NOT NULL -- 'Y'/'N'
);
```

```
INSERT INTO dbo.Products_ECOM (SKU, Title, Brand, Category, Price, ActiveFlag)
VALUES
('LG55-TV', '55 inch LG LED Television', 'LG', 'Television', 689.99, 'Y'),
('SMSGALAXY', 'Samsung Galaxy Smartphone', 'Samsung', 'Mobiles', 479.99, 'Y');
GO
```

```
-- CRM Source
```

```
CREATE DATABASE CRMDB;
```

```
GO
```

```
USE CRMDB;
```

```
GO
```

```
CREATE TABLE dbo.Products_CRM (
    CRMProductID INT IDENTITY(1,1) PRIMARY KEY,
    ProductCode  VARCHAR(50) NOT NULL,
    Name         VARCHAR(200) NOT NULL,
    Brand        VARCHAR(100) NULL,
    Segment      VARCHAR(50) NULL,
    StandardPrice DECIMAL(18,2) NULL,
    Status       VARCHAR(10) NOT NULL -- 'Active'/'Inactive'
);
```

```
INSERT INTO dbo.Products_CRM (ProductCode, Name, Brand, Segment, StandardPrice, Status)
VALUES
('LG55TV', 'LG 55 LED', 'LG', 'TV', 705.00, 'Active'),
('IPHONE', 'iPhone 15', 'Apple', 'MOBILE', 999.00, 'Active');
GO
```

2.2 Landing / Integration Staging DB

```
CREATE DATABASE ProductIntegration;
```

```
GO
```

```
USE ProductIntegration;
```

```
GO
```

```
-- Consolidated landing from all sources
```

```
CREATE TABLE dbo.Landing_Product (
    LandingID      INT IDENTITY(1,1) PRIMARY KEY,
    SourceSystem    VARCHAR(20) NOT NULL,
    SourceProductKey VARCHAR(50) NOT NULL,
    SKU            VARCHAR(50) NOT NULL,
    ProductName     VARCHAR(200) NOT NULL,
    BrandName       VARCHAR(100) NULL,
    CategoryRaw     VARCHAR(50) NULL,
    ListPriceRaw    DECIMAL(18,2) NULL,
    IsActiveRaw     BIT NULL,
    LoadDT        DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);
GO
```

2.3 “SSIS Source” SQL (what you use inside data flows)

ERP extract query:

```
SELECT
    'ERP' AS SourceSystem,
    CAST(ERPProductID AS VARCHAR(50)) AS SourceProductKey,
    SKU,
    ProductName,
    BrandName,
    CategoryCode AS CategoryRaw,
    ListPrice AS ListPriceRaw,
    IsActive AS IsActiveRaw
FROM ERPDB.dbo.Products_ERP;
```

E-commerce extract query:

```
SELECT
    'ECOM' AS SourceSystem,
    CAST(ECOMProductID AS VARCHAR(50)) AS SourceProductKey,
    SKU,
    Title AS ProductName,
    Brand AS BrandName,
    Category AS CategoryRaw,
    Price AS ListPriceRaw,
    CASE WHEN ActiveFlag = 'Y' THEN 1 ELSE 0 END AS IsActiveRaw
FROM ECOMDB.dbo.Products_ECOM;
```

CRM extract query:

```
SELECT
    'CRM' AS SourceSystem,
    CAST(CRMProductID AS VARCHAR(50)) AS SourceProductKey,
    ProductCode AS SKU,
    Name AS ProductName,
    Brand AS BrandName,
    Segment AS CategoryRaw,
    StandardPrice AS ListPriceRaw,
    CASE WHEN Status = 'Active' THEN 1 ELSE 0 END AS IsActiveRaw
FROM CRMDB.dbo.Products_CRM;
```

In SSIS you’d have 3 data flows that all load into ProductIntegration.dbo.Landing_Product.

3 MDS Staging & Business Master Tables

 Real MDS installs create their own stg tables with many required columns.

Below I give a **simplified pattern** that is easy to understand and close to the real thing.

When you work on a real project you would adjust to the exact MDS-created schema.

3.1 Simplified MDS Staging Table for Products

```
USE ProductIntegration;
GO

CREATE TABLE stg.Product_Leaf (
    StagingID    INT IDENTITY(1,1) PRIMARY KEY,
    BatchTag     VARCHAR(50) NOT NULL,
    ImportType    INT NOT NULL, -- 0 = new, 1 = update, 2 = delete (similar idea to MDS)
    Code         VARCHAR(50) NOT NULL, -- Business key / SKU
```

```

Name      VARCHAR(200) NOT NULL,
Brand      VARCHAR(100) NULL,
CategoryCode  VARCHAR(50) NULL,
ListPrice  DECIMAL(18,2) NULL,
IsActive   BIT NOT NULL,
LoadDT     DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
ProcessedFlag BIT NOT NULL DEFAULT 0,
ErrorMessage VARCHAR(500) NULL
);
GO

```

3.2 Example Procedure to Load Staging from Landing (SSIS Execute SQL Task)

```

CREATE PROCEDURE stg.usp_Load_Product_Leaf_From_Landing
    @BatchTag VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO stg.Product_Leaf
    (
        BatchTag, ImportType, Code, Name, Brand,
        CategoryCode, ListPrice, IsActive
    )
    SELECT
        @BatchTag AS BatchTag,
        0 AS ImportType,           -- treat all as new in this simple demo
        SKU AS Code,
        ProductName AS Name,
        BrandName AS Brand,
        CategoryRaw AS CategoryCode,
        ListPriceRaw AS ListPrice,
        COALESCE(IsActiveRaw, 1) AS IsActive
    FROM dbo.Landing_Product
    WHERE LoadDT >= DATEADD(DAY, -1, SYSDATETIME()); -- example filter
END;
GO

```

In SSIS you would:

1. Run data flows to fill Landing_Product.
2. Use **Execute SQL Task**:
3. EXEC stg.usp_Load_Product_Leaf_From_Landing @BatchTag = 'Daily_2025_01_05';

3.3 Simulating “MDS Import” to a Master Table

For lab/demo you can maintain your own master table to mimic MDS result:

```

CREATE TABLE mdm.ProductMaster (
    ProductMasterID INT IDENTITY(1,1) PRIMARY KEY,
    Code      VARCHAR(50) NOT NULL UNIQUE,
    Name      VARCHAR(200) NOT NULL,
    Brand     VARCHAR(100) NULL,
    CategoryCode  VARCHAR(50) NULL,
    ListPrice  DECIMAL(18,2) NULL,
    IsActive   BIT NOT NULL,
    EffectiveFrom  DATETIME2 NOT NULL DEFAULT SYSDATETIME(),
    EffectiveTo    DATETIME2 NULL,
    VersionName    VARCHAR(50) NOT NULL,
    IsCurrent       BIT NOT NULL DEFAULT 1
)

```

```
);
GO
```

Procedure to “import” from staging (similar to what MDS internal procedures do):

```
CREATE PROCEDURE mdm.usp_Import_Product_From_Staging
    @BatchTag VARCHAR(50),
    @VersionName VARCHAR(50)
AS
BEGIN
    SET NOCOUNT ON;

    -- Simple: just insert; in real world you'd handle updates/deletes, SCD, etc.
    INSERT INTO mdm.ProductMaster
    (
        Code, Name, Brand, CategoryCode,
        ListPrice, IsActive, VersionName
    )
    SELECT
        Code, Name, Brand, CategoryCode,
        ListPrice, IsActive, @VersionName
    FROM stg.Product_Leaf
    WHERE BatchTag = @BatchTag
        AND ProcessedFlag = 0;

    UPDATE stg.Product_Leaf
    SET ProcessedFlag = 1
    WHERE BatchTag = @BatchTag;
END;
GO
```

SSIS Execute SQL Task for import:

```
EXEC mdm.usp_Import_Product_From_Staging
    @BatchTag = 'Daily_2025_01_05',
    @VersionName = 'ProductMaster_2025_01';
```

Data Warehouse Table (DimProduct) & Load Script

4.1 DimProduct Table

```
CREATE DATABASE DWH;
GO
USE DWH;
GO

CREATE TABLE dbo.DimProduct (
    ProductKey INT IDENTITY(1,1) PRIMARY KEY,
    ProductCode VARCHAR(50) NOT NULL,
    ProductName VARCHAR(200) NOT NULL,
    BrandName VARCHAR(100) NULL,
    CategoryCode VARCHAR(50) NULL,
    CurrentPrice DECIMAL(18,2) NULL,
    IsActive BIT NOT NULL,
    VersionName VARCHAR(50) NOT NULL,
    SourceLoadDT DATETIME2 NOT NULL DEFAULT SYSDATETIME()
);
GO
```

4.2 Simple Load from Master to DWH (SSIS or SQL job)

```

INSERT INTO DWH.dbo.DimProduct
(
    ProductCode, ProductName, BrandName,
    CategoryCode, CurrentPrice, IsActive, VersionName
)
SELECT
    Code,
    Name,
    Brand,
    CategoryCode,
    ListPrice,
    IsActive,
    VersionName
FROM ProductIntegration.mdm.ProductMaster
WHERE IsCurrent = 1;

```

In a real SSIS package you might:

- Use a **Lookup** to detect new vs existing products.
- Implement SCD type 2 logic (start/end dates, IsCurrent flag).

5 Example Business Rules Reflected in SQL/MDS

You can show both MDS configuration and equivalent SQL:

1. **Code (SKU) must be unique**
 - In MDS: define Code as a code/identifier.
 - In SQL: UNIQUE constraint on mdm.ProductMaster.Code.

2. **ListPrice > 0 for active products**

```

ALTER TABLE mdm.ProductMaster
ADD CONSTRAINT CK_ProductMaster_ListPrice_Positive
CHECK (
    (IsActive = 0 AND ListPrice >= 0)
    OR (IsActive = 1 AND ListPrice > 0)
);

```

3. **Category cannot be NULL for active products**

```

ALTER TABLE mdm.ProductMaster
ADD CONSTRAINT CK_ProductMaster_Category_NotNull_IfActive
CHECK (
    IsActive = 0 OR CategoryCode IS NOT NULL
);

```

These mirror what you'd also configure as **MDS Business Rules**.

6 How to Describe This as an Interview “Project Story”

Here's a full narrative you can adapt.

(Think of this as a 3–5 minute answer to “Tell me about your MDS project.”)

◆ Project Context

I worked on a **Product Master Data Management** project for a mid-sized retail organization called *RetailX*.

They were selling electronics through ERP, e-commerce, and CRM systems, each with its own product database.

Because of that, the same product existed with different SKUs, names, and prices across systems, which caused pricing errors, wrong shipments, and inconsistent reports.

◆ Your Role

I was the **SQL Server / MDS developer** responsible for designing the MDS model, staging area, SSIS integrations, and business rules.

◆ What You Designed

We implemented **SQL Server Master Data Services** as the central hub for product master data.

- I designed a **ProductMaster model** with entities like Product, Brand, and Category.
- The Product entity had attributes such as Code (SKU), Name, Brand, Category, ListPrice, and IsActive.
- We created a **derived hierarchy** Category > Product for reporting and navigation.

◆ Data Flow / Architecture

From an architecture perspective, we built a separate **Integration/Staging database**.

- Using **SSIS**, we extracted product data from three systems: ERP, E-com, and CRM.
- In the SSIS packages, we standardized the columns and loaded them into a consolidated Landing_Product table.
- From there, I built a stored procedure stg.usp_Load_Product_Leaf_From_Landing to populate the **MDS staging table** stg.Product_Leaf with a BatchTag.
- Another procedure, mdm.usp_Import_Product_From_Staging, simulated the MDS import into our master table mdm.ProductMaster with versioning, which in real MDS is handled by the MDS engine.

Once the data was imported and validated in MDS, we created **subscription views** / master views and used SSIS to load the **DimProduct** table in the data warehouse.

◆ Business Rules & Data Quality

A key part of my work was **data quality and business rules**:

- Code (SKU) had to be unique and not null.
- Active products must have a positive price, enforced both with MDS business rules and SQL check constraints.
- No active product was allowed without a valid category.
- When a product was marked inactive, we automatically set an inactive date, using change-based rules in MDS.

Data stewards used the **MDS web portal and Excel add-in** to review exceptions, fix invalid records, and approve changes before we committed each version.

◆ Versioning & Deployment

We used **monthly versions** for the Product model, for example ProductMaster_2025_01.

The process was:

1. Import data into an open version.
2. Validate using business rules and resolve issues.
3. Lock and commit the version once approved.
4. Use that committed version as the **source of truth** for downstream systems.

◆ Outcome / Business Value

After rollout, the company had **one canonical product view** across ERP, CRM, and reporting.

We eliminated a large number of duplicate SKUs, reduced pricing discrepancies on the website, and reporting became consistent because the data warehouse was now fed from the **MDS golden copy**.

Business users appreciated the Excel add-in because they could maintain products themselves while still complying with governance.

◆ Tech Stack Keywords to Mention

SQL Server, Master Data Services, SSIS, T-SQL, staging / landing design, business rules, hierarchies, versioning, DimProduct, data warehouse, data governance.

Practical's to visualize the SSIS packages and quick revision checklist before interviews.

1 SSIS Package Design – Step-by-Step (Control Flow + Data Flow)

We'll design 3 main SSIS packages, matching the logical flow we discussed:

1. PKG_Extract_And_Land_Products
2. PKG_Load_MDS_Staging_Products
3. PKG_Load_DWH_DimProduct_From_MDM

You can mention these *exact names* in interviews.

Package 1: PKG_Extract_And_Land_Products

Goal: Extract products from ERP, ECOM, CRM → load into ProductIntegration.dbo.Landing_Product.

A. Control Flow

1. **Execute SQL Task – Truncate Landing Table (Optional but common)**
 - Name: TRUNCATE Landing_Product
 - Connection: ProductIntegration
 - SQL:
 - TRUNCATE TABLE dbo.Landing_Product;
2. **Data Flow Task – Load ERP Products**
 - Name: DFT_Load_ERP_Products
3. **Data Flow Task – Load ECOM Products**
 - Name: DFT_Load_ECOM_Products
4. **Data Flow Task – Load CRM Products**
 - Name: DFT_Load_CRM_Products

(You can run them sequentially or in **parallel** to show performance awareness.)

B. Data Flow: DFT_Load_ERP_Products

Source → Transform → Destination pattern

1. **OLE DB Source**
 - Connection: ERPDB
 - SQL Command:


```
SELECT
    'ERP' AS SourceSystem,
    CAST(ERPProductID AS VARCHAR(50)) AS SourceProductKey,
    SKU,
    ProductName,
    BrandName,
    CategoryCode AS CategoryRaw,
    ListPrice AS ListPriceRaw,
    IsActive AS IsActiveRaw
FROM ERPDB.dbo.Products_ERP;
```
2. **Derived Column Transformation (standardization)**
 - Columns:
 - SKU → LTRIM(RTRIM(SKU))
 - ProductName → UPPER(ProductName) (optional)
 - BrandName → UPPER(BrandName) (optional)
 - CategoryRaw → UPPER(CategoryRaw)
3. **OLE DB Destination**
 - Connection: ProductIntegration
 - Table: dbo.Landing_Product
 - Mapping: all columns to the corresponding fields.

C. Data Flow: DFT_Load_ECOM_Products

1. **OLE DB Source**
 - Connection: ECOMDB
 - SQL:


```

SELECT
    'ECOM' AS SourceSystem,
    CAST(ECOMProductID AS VARCHAR(50)) AS SourceProductKey,
    SKU,
    Title AS ProductName,
    Brand AS BrandName,
    Category AS CategoryRaw,
    Price AS ListPriceRaw,
    CASE WHEN ActiveFlag = 'Y' THEN 1 ELSE 0 END AS IsActiveRaw
FROM ECOMDB.dbo.Products_ECOM;

```

2. **Derived Column** (similar cleanup)
3. **OLE DB Destination** → ProductIntegration.dbo.Landing_Product

D. Data Flow: DFT_Load_CRM_Products

1. **OLE DB Source**
 - Connection: CRMDB
 - SQL:

```

SELECT
    'CRM' AS SourceSystem,
    CAST(CRMProductID AS VARCHAR(50)) AS SourceProductKey,
    ProductCode AS SKU,
    Name AS ProductName,
    Brand AS BrandName,
    Segment AS CategoryRaw,
    StandardPrice AS ListPriceRaw,
    CASE WHEN Status = 'Active' THEN 1 ELSE 0 END AS IsActiveRaw
FROM CRMDB.dbo.Products_CRM;

```

2. **Derived Column**
3. **OLE DB Destination** → Landing_Product

Package 2: PKG_Load_MDS_Staging_Products

Goal: Move consolidated data from Landing_Product → stg.Product_Leaf (simulating MDS staging load).

A. Parameters (good for interviews)

- User::BatchTag (String) – e.g., "Daily_" + YYYYMMDD

B. Control Flow

1. **Execute SQL Task – Call Staging Load Proc**
 - Name: EXEC usp_Load_Product_Leaf_From_Landing
 - Connection: ProductIntegration
 - SQL:

```
EXEC stg.usp_Load_Product_Leaf_From_Landing @BatchTag = ?
```
 - Map parameter:
 - @BatchTag ← User::BatchTag
2. **Execute SQL Task – Call Import Proc (MDS-like)**
 - Name: EXEC usp_Import_Product_From_Staging
 - SQL:

```
EXEC mdm.usp_Import_Product_From_Staging
    @BatchTag = ?,
    @VersionName = ?;
```
 - Parameters:
 - @BatchTag ← User::BatchTag
 - @VersionName ← User::VersionName (another package parameter, e.g. "ProductMaster_2025_01")
3. **(Optional) Execute SQL Task – Validation / Log**
 - Query row counts or errors from staging and log them in a log table.

Package 3: PKG_Load_DWH_DimProduct_From_MDM

Goal: Load **DimProduct** from mdm.ProductMaster (only current, trusted data).

A. Control Flow

1. **Data Flow Task – Load DimProduct**
 - Name: DFT_Load_DimProduct

B. Data Flow: DFT_Load_DimProduct

There are two levels: simple (insert-only) and advanced (SCD). I'll give both concepts.

Option 1: Simple Insert-Only (Good Enough for Demo)

1. **OLE DB Source**
 - Connection: ProductIntegration
 - SQL:

```
SELECT
    Code,
    Name,
    Brand,
    CategoryCode,
    ListPrice,
    IsActive,
    VersionName
FROM mdm.ProductMaster
WHERE IsCurrent = 1;
```
2. **OLE DB Destination**
 - Connection: DWH
 - Destination table: dbo.DimProduct
 - Mapping fields to ProductCode, ProductName, etc.

Option 2: SCD Type 2 (Great to mention in interviews)

1. **OLE DB Source**
 - Same SQL as above.
2. **Lookup Transformation**
 - Connect to DWH.dbo.DimProduct
 - Match on ProductCode
 - Redirect rows to:
 - **Matched Output** → potential updates
 - **No Match Output** → new inserts
3. **Conditional Split**
 - For matched rows, compare key columns (e.g., price, category):
 - If changed → go to **"Changed Rows"**
 - Else → **"Unchanged Rows"** (drop or ignore)
4. **OLE DB Command** (for changed rows)
 - Set IsActive = 0 or mark old record as historical (if you modeled SCD with dates).
5. **OLE DB Destination** – New row insert
 - Inserts new versions into DimProduct (either all from NoMatch, or from changed rows after update).

Even if you haven't fully built SCD in practice, **being able to explain this flow clearly is a big plus.**

Quick MDS + SSIS Revision Checklist (Pre-Interview)

Use this like a **10-minute revision sheet**.

Core MDS Concepts

- What is Master Data? Examples: Product, Customer, Vendor, Chart of Accounts.
- What is MDS? (Master Data Services – MDM tool in SQL Server).
- Key components:
 - Model

- Entity
- Attributes
- Members
- Hierarchies (explicit / derived)
- Versions
- Business Rules
- Staging tables
- Subscription views
- Domain-based attributes (relationship between entities).
- Difference between **leaf members**, **consolidated members** (in real MDS).
- What is versioning? Why commit/freeze?

Data Quality & Business Rules

- How to enforce:
 - Unique SKU
 - Mandatory fields (Name, Category)
 - Price > 0 for active products
- Change-based rules (e.g., setting InactiveDate when IsActive becomes false).
- Validation process in MDS (validate members/version, fix, revalidate).

SSIS + MDS Integration

- Typical pattern:
 1. Extract from **source systems**
 2. Load to **Landing / Integration DB**
 3. Load to **MDS staging tables** (stg schema)
 4. Execute **MDS import stored procedures**
 5. Validate in MDS (manual + rules)
 6. Use **subscription views** to export master data
 7. Load **DWH dimensions** (e.g., DimProduct)
- Components you should be comfortable speaking about:
 - OLE DB Source / Destination
 - Derived Column
 - Lookup
 - Conditional Split
 - Execute SQL Task
 - Parameters and Configurations

Data Warehouse Side

- Purpose of **DimProduct** / **DimCustomer**.
- Simple vs SCD Type 2:
 - Why track history?
 - IsCurrent flag / StartDate / EndDate approach.

Project Story Essentials

Be ready to answer:

- “Tell me about your MDS project.”
- “What problem did MDS solve?”
- “How did you integrate MDS with upstream and downstream systems?”
- “What kind of business rules did you enforce?”
- “How did you handle duplicates and conflicting records?”
- “How did business users interact with MDS?” (Web UI, Excel add-in)
- “What were the limitations of MDS in your project?”

Below is a **full end-to-end mock interview simulation**, covering **MDS, SSIS, ETL, data governance, architecture**, and **real-world troubleshooting**.

★ MASTER DATA SERVICES (MDS) — FULL MOCK INTERVIEW ★

◆ SECTION 1 — BASIC INTRODUCTION

Interviewer:

Tell me about your experience with SQL Server Master Data Services.

Candidate (ideal answer):

I worked on a full Master Data Management initiative where we implemented SQL Server MDS to centralize **Product, Brand**, and **Category** master data across multiple systems: SAP ERP, Salesforce CRM, and an e-commerce platform.

My responsibilities included:

- Designing **MDS models, entities, attributes, hierarchies**
- Creating **business rules** for data quality
- Building **SSIS pipelines** to integrate source systems → MDS staging → MDS model
- Publishing subscription views to downstream systems
- Implementing **versioning, validation**, and **workflow** processes for business stewards

The project reduced duplicate products by ~35% and standardized product naming and hierarchies across departments.

◆ SECTION 2 — MDS FUNDAMENTALS + ARCHITECTURE

Interviewer:

What is MDS and why do companies use it?

Candidate:

MDS is Microsoft's Master Data Management component inside SQL Server.

It provides:

- A **centralized repository** for critical business entities
- **Data quality rules** & governance workflows
- **Hierarchies** and relationships
- Versioning & auditing
- Integration interfaces (staging tables, subscription views)

Companies use MDS to create **trusted, standardized master data** for analytics, ERP, CRM, and other applications.

Interviewer:

Explain the architecture you implemented.

Candidate:

Our architecture had four major layers:

1. Upstream systems

SAP (ERP), Salesforce (CRM), and an e-commerce DB.

2. Integration/Staging Layer (SQL + SSIS)

- Raw extractions land into a **Landing DB**
- Transformations standardize and map data
- Data moves to **MDS staging tables** (stg schema)

3. MDS Layer

- Model: ProductMaster
- Entities: Product, Brand, Category
- Attributes, hierarchies, business rules
- Versioning: monthly committed versions

4. Downstream Consumption

- Subscription views (mdm schema)
- Data warehouse load (DimProduct)
- API-based export to e-commerce systems
- Power BI reporting

This architecture ensured **end-to-end governance, traceability**, and **high-quality master data**.

◆ SECTION 3 — MODEL, ENTITIES, ATTRIBUTES

Interviewer:

Explain how you designed your Product entity in MDS.

Candidate:

The Product entity included:

Attribute	Type	Purpose
Code	Identifier	SKU / business key
Name	Text	Product description
Brand	Domain-based	Linked to Brand entity
Category	Domain-based	Linked to Category hierarchy
ListPrice	Number	Pricing
IsActive	Boolean	Current status
EffectiveStartDate	Date	SCD tracking
EffectiveEndDate	Date	SCD tracking

We used domain-based attributes so we could maintain **Brand** and **Category** separately and enforce data consistency.

◆ SECTION 4 — BUSINESS RULES

Interviewer:

What business rules did you configure in MDS?

Candidate:

I configured multiple rules:

1. **Mandatory Fields** – Name, Brand, Category
2. **Duplicate Prevention** – SKU cannot be repeated
3. **Price Rules**
 - If Product is Active → ListPrice must be > 0
4. **Default Category Assignment**
 - If Category is NULL → assign “UNASSIGNED” and create a warning
5. **Automatic Field Updates**
 - If IsActive changes from TRUE to FALSE → set EndDate to Today()

These rules helped automate validation before committing a version.

◆ SECTION 5 — MDS STAGING & IMPORT

Interviewer:

How did you load data into MDS?

Candidate:

We used **MDS staging tables** such as:

- stg.Product_Leaf
- stg.Brand_Leaf
- stg.Category_Leaf

The process:

1. SSIS loads data into staging tables
2. Execute stored procedures like:
3. EXEC stg.udp_Product_Leaf @VersionName = 'ProductMaster_2024_05';
4. MDS processes inserts/updates/deletes
5. Business users validate via Web UI
6. Commit version when complete

This staging load approach is scalable and fully automatable.

◆ SECTION 6 — SUBSCRIPTION VIEWS (EXPORT)

Interviewer:

How did you export MDS data downstream?

Candidate:

We created MDS subscription views like:

- mdm.sv_ProductMaster
- mdm.sv_CategoryHierarchy

These views always return **the latest validated data**, making it easy for SSIS to load into:

- Data warehouse (DimProduct)
- E-commerce DB
- API exports
- Power BI datasets

We scheduled nightly loads via SQL Agent.

◆ SECTION 7 — SSIS INTEGRATION & ETL PROCESS

Interviewer:

Explain your SSIS workflow from source systems to MDS and then to the DWH.

Candidate:

We had a three-layer ETL pipeline:

1. Extract Packages

- Pull from SAP, Salesforce, ECOM
- Standardize text, trim spaces, fix cases
- Load into Landing_Product table

2. MDS Load Package

- Load from Landing DB → stg.Product_Leaf
- Run staged MDS stored procedures
- Log errors / mismatches
- Notify data stewards if validation fails

3. DWH Load Package

- Consume subscription views
- Perform SCD Type 2 processing
- Load DimProduct and DimCategory

This created a **fully governed data supply chain**.

◆ SECTION 8 — HIERARCHIES

Interviewer:

How did you implement hierarchies in MDS?

Candidate:

We implemented a **derived hierarchy**:

Category → SubCategory → Product

Steps:

1. Built Category entity with ParentCategory attribute
2. Configured a derived hierarchy using the parent-child relationship
3. Assigned each Product to a leaf in the hierarchy
4. Applied rules ensuring that each Product must belong to only one Category

These hierarchies were used by analysts and finance teams for reporting.

◆ SECTION 9 — VERSIONING & WORKFLOWS

Interviewer:

How did you handle MDS versioning?

Candidate:

We implemented monthly versions:

1. Create new version
2. Import source system data
3. Apply business rules
4. Data stewards validate and correct records
5. Once complete → **Commit Version**
6. Subscription views automatically refresh from latest committed version

This ensured controlled, auditable releases of master data.

◆ SECTION 10 — REAL-TIME TROUBLESHOOTING

Interviewer:

What issues did you face loading data into MDS?

Candidate:

Common issues included:

1. **Failed business rule validations**
 - Resolved by updating staging data or adjusting rules
2. **Duplicate SKU conflicts**
 - Fixed using match/merge logic in SSIS
3. **Domain-based attribute mismatches**
 - Occurred when Brand or Category did not exist yet
 - Solved by loading lookup entities first
4. **Invalid hierarchies**
 - Circular parent-child references
5. **Performance issues** during bulk imports
 - Optimized by batching loads and indexing staging tables

These troubleshooting skills are critical in real MDM environments.

◆ SECTION 11 — COMPARISON QUESTIONS

Interviewer:

Compare MDS with Azure Purview and Profisee.

Candidate:

Feature	MDS	Azure Purview	Profisee
Type	MDM system	Data catalog	Enterprise MDM
Rules	Yes	No	Yes (advanced)
Integration	SSIS + staging	Lineage only	APIs + workflows
Cloud support	No (on-prem)	Azure-native	Hybrid

- MDS = good for on-prem SQL Server shops
- Purview = metadata + lineage, not MDM
- Profisee = full enterprise-level MDM with workflow + match/merge + golden record creation

◆ SECTION 12 — SCENARIO QUESTIONS

Interviewer:

If a product appears in three source systems with different names and prices, how do you determine the golden record?

Candidate:

We implemented a **priority-based survivorship rule**:

1. SAP = authoritative for price
2. Salesforce = authoritative for descriptions
3. ECOM = authoritative for active flag
4. Use SSIS transformations to merge inputs
5. Load the final merged record into MDS staging
6. Let MDS rules validate final entries
7. Publish golden record to DWH

This ensures consistent and repeatable golden record creation.

◆ SECTION 13 — FINAL QUESTION

Interviewer:

Why should we trust your MDS implementation skills?

Candidate:

Because I've delivered a real-world MDM system where:

- Data quality improved significantly
- Data stewards gained better control

- Duplicate entries were eliminated
- Reporting consistency increased
- ETL and master data governance were automated end-to-end

Understand MDS not only technically, but also from a **business governance** perspective.

Below is a **complete rapid-fire round of 50 Master Data Services (MDS) interview questions**—fast, punchy, and perfect for last-minute preparation.

⚡ 50 Rapid-Fire SQL Server Master Data Services (MDS) Interview Questions

✅ MDS Fundamentals

1. What is SQL Server Master Data Services (MDS)?
2. What business problem does MDS solve?
3. What are Models in MDS?
4. What is an Entity in MDS?
5. What is the difference between Leaf, Consolidated, and Collection members?
6. What are Attributes in MDS?
7. What are domain-based attributes?
8. What is an attribute group?
9. What is a hierarchy in MDS?
10. Difference between Explicit and Derived hierarchies?

✅ Versioning & Data Governance

11. What are Versions in MDS?
12. What is VersionFlag?
13. What is the purpose of version validation?
14. How do you lock a version?
15. What is the workflow approval process in MDS?
16. How do you handle Type-2 history in MDS?

✅ Business Rules & Validation

17. What are Business Rules in MDS?
18. Can business rules trigger notifications?
19. What are the most common rule types used in MDS projects?
20. Can business rules update attribute values automatically?
21. What is the difference between Validation Issue and Validation Status?

✅ Security

22. What types of security are available in MDS?
23. Difference between Model Object permissions and Attribute permissions?
24. What is Hierarchy Member Security?
25. How do you give read-only access to an MDS user?

✅ Integration (SSIS / API / Staging)

26. What is the purpose of MDS staging tables?
27. What is the difference between stg.Member and stg.Entity staging tables?
28. What is the purpose of the "ImportType" column in staging tables?
29. How do you trigger a staging batch?
30. What are subscription views?
31. When do you use tblModel, tblEntity, tblAttribute tables?
32. What is MDS Web Service used for?
33. What are the challenges integrating MDS with SAP/CRM systems?

✓ Performance & Architecture

34. How do you optimize MDS performance?
35. How does MDS store metadata internally?
36. What is the difference between MDS database and MDS application?
37. How does MDS handle large hierarchies?
38. When would you *not* recommend using MDS?

✓ Design & Modeling

39. What considerations go into designing an MDM model?
40. How do you enforce uniqueness for an attribute in MDS?
41. When should you use consolidated members?
42. What are the common pitfalls in designing product master models?
43. How do you model reference data vs master data in MDS?

✓ Deployment & Operations

44. How do you deploy an MDS model from DEV to PROD?
45. What is the purpose of the MDSModelDeploy tool?
46. How do you backup and restore MDS environments?
47. What types of logs does MDS generate?
48. How do you troubleshoot failed staging batches?
49. How do you handle user-driven corrections?
50. What monitoring should be implemented for production MDS?

Below are **detailed, interview-quality answers** to all **50 rapid-fire SQL Server Master Data Services (MDS)** questions. These answers are crafted exactly how a strong candidate would speak in an interview—concise but confident.

⚡ MASTER DATA SERVICES — 50 QUESTIONS WITH ANSWERS

✓ MDS Fundamentals

1. What is SQL Server Master Data Services (MDS)?

MDS is Microsoft's **Master Data Management (MDM)** platform used to define, manage, govern, and publish **trusted master data** such as Products, Customers, Vendors, etc.

2. What business problem does MDS solve?

It eliminates **duplicate, inconsistent, or inaccurate** master data across systems, creating a **single source of truth**.

3. What are Models in MDS?

A Model is the **highest-level container** that groups entities, attributes, hierarchies, and business rules related to a domain (e.g., Product Model).

4. What is an Entity in MDS?

An Entity is like a **table** that stores master data records—e.g., Product, Customer, Brand.

5. Difference between Leaf, Consolidated, and Collection members?

- **Leaf** — actual master data records.
- **Consolidated** — used to group leaf members into hierarchies.
- **Collections** — user-defined groupings not tied to hierarchy.

6. What are Attributes in MDS?

Attributes are **columns** of an entity (Name, Brand, Price, etc.).

7. What are domain-based attributes?

Attributes that reference **other entities**—like foreign keys.

8. What is an attribute group?

A UI grouping of attributes used to simplify data entry pages.

9. What is a hierarchy in MDS?

A structured representation of how entities relate, often used for reporting or data governance.

10. Difference between Explicit and Derived hierarchies?

- **Explicit** — manually defined using consolidated members.
- **Derived** — automatically created from **domain-based attributes**.

 Versioning & Data Governance**11. What are Versions in MDS?**

A snapshot of master data used for lifecycle management, approvals, and audits.

12. What is VersionFlag?

A label (e.g., “Current”, “Latest”) used to reference the correct version programmatically.

13. What is the purpose of version validation?

To ensure data meets all **business rules** before the version can be locked or published.

14. How do you lock a version?

Once validated, you set the version to **Locked**, preventing data edits.

15. What is the workflow approval process in MDS?

A SQL Server workflow integration where changes trigger **notification**, review, and approval by data stewards.

16. How do you handle Type-2 history in MDS?

MDS stores historical versions automatically. Each version becomes a Type-2 history snapshot.

 Business Rules & Validation**17. What are Business Rules in MDS?**

Logic that enforces data quality—e.g., required fields, value constraints, default values.

18. Can business rules trigger notifications?

Yes. Business rules can send **email notifications** when validation fails.

19. Common rule types used in MDS projects?

- Attribute required
- Value must be in range
- Auto-default a value
- Reference entity validation
- Conditional rules

20. Can business rules update attribute values automatically?

Yes—business rules can **set values**, default values, or copy values.

21. Difference between Validation Issue and Validation Status?

- **Issue**: Specific rule violation.
- **Status**: Whether the record is VALID or INVALID.

 Security**22. What types of security are available in MDS?**

- **Functional** (UI pages/features)
- **Model Object** (entity-level permissions)
- **Member Security** (record-level security)

- **Attribute Security** (column-level)

23. Difference between Model Object permissions and Attribute permissions?

- **Model Object** controls access to entire entity.
- **Attribute** controls access to **specific columns**.

24. What is Hierarchy Member Security?

Row-level security applied to nodes in a hierarchy.

25. How do you give read-only access to an MDS user?

Assign **Read** permissions at:

- Model
 - Entity
 - Attribute
- Optionally deny update/delete.

✓ Integration (SSIS / API / Staging)

26. Purpose of MDS staging tables?

To load data into MDS from ETL/SSIS before it is validated and merged into core tables.

27. Difference between stg.Member and stg.Entity staging tables?

- **stg.Member** → used for leaf/consolidated members
- **stg.Entity** → used for entity attribute updates

28. Purpose of ImportType column?

Controls mode:

- 0 = Create only
- 1 = Create & Update
- 2 = Delete
- 3 = Soft-delete

29. How do you trigger a staging batch?

Insert rows into staging tables → Insert into **stg.Batch** → Use stored procedure **udp_StageProcess**.

30. What are subscription views?

SQL views automatically maintained by MDS to publish **clean master data** to downstream systems.

31. When do you use tblModel, tblEntity, tblAttribute tables?

They are internal **metadata tables**—rarely used directly except for metadata queries.

32. What is MDS Web Service used for?

Programmatic access to MDS—CRUD operations, workflows, versioning, etc.

33. Challenges integrating MDS with SAP/CRM systems?

- Duplicate keys
- Mapping to different vocabularies
- Slow delta captures
- Data quality issues
- Handling high volumes in staging

✓ Performance & Architecture

34. How do you optimize MDS performance?

- Index staging tables
- Archive old versions

- Reduce complex hierarchies
- Optimize business rules
- Scale IIS + SQL Server resources

35. How does MDS store metadata internally?

Normalized metadata tables:

- **mdm.tblModel**
- **mdm.tblEntity**
- **mdm.tblAttribute**

Plus leaf tables storing attribute values separately.

36. Difference between MDS database and MDS application?

- **Database:** stores master data + metadata
- **Application:** UI, web services, workflows, security

37. How does MDS handle large hierarchies?

Uses **recursive SQL structures**, indexed hierarchy tables, and batch operations.

38. When would you *not* recommend using MDS?

- Very high-volume master data (>20M rows)
- Real-time MDM scenarios
- When advanced DQ, match/merge is required (use Profisee, Informatica, Reltio instead)



Design & Modeling

39. Considerations when designing an MDM model?

- Entity granularity
- Attribute naming & datatype
- Business rules
- Ownership & governance
- Workflow
- Hierarchy needs
- Integration architecture

40. How do you enforce uniqueness for an attribute in MDS?

Define it as a **Code**, or use a **Business Rule** to reject duplicates.

41. When should you use consolidated members?

When creating an **explicit hierarchy**, like Category → Subcategory → Product.

42. Common pitfalls in designing product master models?

- Too many attributes
- Unoptimized domain-based attributes
- Overuse of explicit hierarchies
- Lack of ownership/governance clarity

43. How do you model reference data vs master data in MDS?

- **Master data** = high-value business entities (Products/Customers)
- **Reference data** = code lists (Status, Country, Currency)



Deployment & Operations

44. How do you deploy an MDS model from DEV to PROD?

Using **MDSModelDeploy.exe** to export a model package and import it to another environment.

45. Purpose of MDSModelDeploy tool?

To ** deploy, export, import, and version** MDS models between environments.

46. How do you backup and restore MDS environments?

Backup the **MDS database**, plus IIS config if needed.

47. What types of logs does MDS generate?

- Staging logs
- Business rule logs
- Service logs
- Workflow logs
- Security logs

48. How do you troubleshoot failed staging batches?

Check:

- stg.Error tables
- mdm.tblStgBatchLog
- Event logs
- Business rule issues
- Invalid domain-based references

49. How do you handle user-driven corrections?

Through:

- MDS Web UI
- Excel Add-In
- Workflow approvals
- Data Steward reviews

50. What monitoring should be implemented for production MDS?

- Staging batch failures
- Business rule failures
- Version validation errors
- IIS health
- SQL performance
- Subscription view refresh success

Below is a **condensed 5-page Master Data Services (MDS) interview cheat sheet** followed by a **resume-ready MDS project description** you can paste directly into your CV.

MASTER DATA SERVICES (MDS) — 5-PAGE INTERVIEW CHEAT SHEET

(Designed for rapid revision before interviews—covers architecture, modeling, integration, governance, SSIS, workflows, troubleshooting, and scenarios.)

PAGE 1 — MDS FUNDAMENTALS & ARCHITECTURE

★ What is MDS?

Microsoft SQL Server Master Data Services (MDS) is an **MDM platform** for storing, managing, validating, and publishing **golden master data** such as Products, Customers, Vendors, Employees, Finance dimensions, etc.

★ Core Capabilities

- Centralized **master data repository**
- **Governance + versioning**
- **Business rules & data quality**
- **Workflow approvals**
- **Hierarchies & relationships**
- **Integration** via staging, subscription views, Excel add-in, API

★ MDS Architecture Components

1 MDS Database (mdm DB)

- Stores metadata, versions, leaf tables, business rules, staging, logs.

2 MDS Web Application (IIS)

- UI for data stewards, workflows, hierarchy editing.

3 MDS Web Service

- SOAP API for integration/automation.

4 Excel Add-In

- Data steward bulk editing.

5 SSIS / Staging Tables

- ETL integration layer.

6 Subscription Views

- Published output (trusted golden data → DWH, CRM, ERP, APIs).

PAGE 2 — MODELING, ENTITIES, ATTRIBUTES & HIERARCHIES

★ MDS Model Design

A Model is a “container” for a domain (e.g., **ProductMaster**).

A model contains:

✓ Entities

Equivalent to tables.

Examples: *Product, Brand, Category, Vendor*.

✓ Attributes

Columns inside entities:

- Free-form attributes (Name, Description, Price)
- Domain-based attributes (Brand → Brand entity)
- Derived attributes (from hierarchies)

✓ Member Types

Member Type Used For

Leaf Actual data records

Consolidated Nodes for explicit hierarchies

Collections User-defined groups

★ Hierarchies

Two types:

1. Derived Hierarchy

Created automatically from **domain-based attributes**.

Example: Product → Category → Department

2. Explicit Hierarchy

Manually created using **Consolidated + Leaf members**.

Used for complex structures like:

- Financial reporting structures
- Custom multi-level category trees

★ Attribute Groups

Logical grouping of attributes in UI—makes stewarding easier.

PAGE 3 — BUSINESS RULES, VERSIONING, WORKFLOWS & SECURITY

★ Business Rules

Enforce data quality and governance:

Common rules:

- Required fields
- Value ranges
- Auto-default values
- Conditional logic
- “If X is blank → set Y”
- Validate domain relationships
- Trigger notifications on failure

Business rule results appear as:

- **ValidationStatus** (VALID / INVALID)
- **ValidationIssues** (detailed errors)

★ Versioning

MDS supports multi-version master data:

- **Open** → editable
- **Validated** → meets rules
- **Locked** → immutable snapshot
- **Committed** → ready to publish

VersionFlag = named bookmark like “CurrentVersion”.

★ Workflows

Uses SQL Server Workflow:

- Triggered by business rules
- Sends notifications
- Allows approval/reject steps
- Tracks data steward actions

★ Security

Four levels of security:

Layer	Description
Functional Security	UI and feature permissions
Model Object Security	Entity-level access
Attribute Security	Column-level access
Hierarchy Member Security	Row-level security

PAGE 4 — INTEGRATION: STAGING, SSIS, SUBSCRIPTION VIEWS

★ The MDS Staging Process

Used for bulk data loads (SSIS, ETL Tools, APIs).

Key staging tables:

- **stg_Leaf** (for record inserts/updates)
- **stg_Attribute** (for attribute updates)
- **stg.Member, stg.Entity** (generic load options)
- **stg.Batch** (batch processing table)

ImportType

Controls the load operation:

- **0 = Create only**
- **1 = Create & Update**
- **2 = Delete**
- **3 = Soft-delete**

Steps:

- 1 Insert into staging tables
- 2 Create batch record
- 3 Execute: udp_StageProcess
- 4 Data validated → stored in mdm tables
- 5 If errors → viewed in stg tables

★ Subscription Views

Pre-built SQL views MDS auto-generates for:

- Data warehouse loads
- Downstream consumption
- Reporting
- Application integration

Types:

- **Leaf members views**
- **Consolidated members views**
- **Hierarchy flattened views**
- **Delta views**

These ensure downstream systems always read **validated, approved master data**.

★ SSIS Integration

Common workflows:

- Load raw data → staging tables
- Validate → update logs
- Extract subscription views → DWH
- Automate workflows & versioning
- Scheduling & monitoring

PAGE 5 — TROUBLESHOOTING, BEST PRACTICES & REAL-WORLD SCENARIOS

★ Troubleshooting Guide

Issue	Cause	Fix
Invalid records	Failed business rules	Review ValidationIssue tables
Failed staging batch	Bad ImportType, missing attributes	Check stg.Error, batch logs
Hierarchy mismatch	Duplicate keys	Clean source data
Slow UI	Too many attributes / rules	Optimize entity design
Integration failing	Domain-based attribute mismatch	Ensure reference data loaded first

★ Best Practices

- Start with **minimal attributes**, expand gradually
- Create **clear ownership** (data stewards, approvers)
- Use **domain-based attributes** for referential integrity
- Keep hierarchies **lean & logical**
- Version frequently → avoid large deltas
- Use **subscription views instead of direct table access**
- Implement **error-handling SSIS patterns**

★ Real-World Scenarios

✓ Scenario 1: Product synchronization between ERP, CRM, and eCommerce

Use MDS to unify Product entity → enforce rules → publish DWH-ready golden product list.

✓ Scenario 2: Customer Master consolidation

Use survivorship rules + workflows + domain attributes.

✓ Scenario 3: Financial reporting hierarchy management

Explicit hierarchies + versioning + workflow approvals.

✓ Scenario 4: Data Stewardship process

Excel Add-In for corrections → business rules → workflow approval → publish version.

🔗 RESUME-READY: MDS PROJECT EXPERIENCE (COPY-PASTE SECTION)

Below is a **polished, ATS-friendly, senior-level resume entry** describing strong MDS experience.

★ Master Data Management (MDM) / SQL Server Master Data Services (MDS) Project

Role: MDM Developer / Data Engineer / BI Engineer

Technologies: SQL Server MDS, SSIS, SQL, Azure Data Factory, DWH, Power BI, Excel Add-In

Project Summary:

Led the implementation of a centralized **Product Master Data Management (MDM) platform** using SQL Server Master Data Services (MDS) to create a unified, high-quality, governed source of Product, Category, Brand, and Vendor data. Consolidated master data from SAP ERP, Salesforce CRM, and eCommerce systems, eliminating inconsistencies, duplicates, and mismatched business hierarchies.

Key Responsibilities & Achievements:

- Designed and implemented the **MDS ProductMaster model** including entities, attributes, domain-based attributes, and hierarchies.
- Built **derived and explicit hierarchies** for Product → Category → Subcategory and Vendor → Region.
- Defined **60+ business rules** ensuring data quality, completeness, referential integrity, and attribute validation.
- Implemented **versioning strategy**, version flags, and automated validation processes.
- Developed **SSIS ETL pipelines** to load data into MDS staging tables, trigger MDS batch processes, manage deltas, and publish subscription views to the enterprise DWH.
- Integrated MDS with downstream systems using **subscription views**, enabling consistent master data across reporting, analytics, and operational systems.
- Implemented **data stewardship workflows** for approvals, notifications, and exception handling.
- Built **error-handling frameworks** for staging batches and validation failures.
- Designed and delivered **governance processes** involving data stewards, SMEs, and business owners.
- Enabled **Excel Add-In** for bulk data entry, corrections, and stewardship.
- Improved data accuracy by **~30%**, reduced duplicate product entries by **40%**, and standardized 100% of product attributes across systems.

Business Impact:

- Delivered a “single source of truth” for product data across the organization.
- Improved reporting accuracy and reduced reconciliation time.
- Enabled consistent pricing, naming, and categorization across ERP, CRM, eCommerce, and analytics.
- Established a scalable MDM foundation for future customer and vendor master initiatives.

<https://www.linkedin.com/in/praveenmadupu/>

<https://github.com/PMSQLDBA>